
A04**Trees & Maps****Algorithmic Thinking****ALD (MTD.ba VZ)****SS 2026**

Submission via Moodle

24 Points

This assignment applies the data structures from Sessions 6 and 7 in two short scenarios. **Let the data structure enforce the rules** — call the right operations instead of re-implementing BST or map logic elsewhere.

Submission Requirements:

- Read the [Organizational Guidelines](#) on Moodle, and adhere to them!
 - Submit a PDF document containing your solution idea, code, and explanations
 - Include all source files in a ZIP archive (extend your UE7 Maven project)
 - Use the naming convention: `ALD_A04_LASTNAME.pdf` and `ALD_A04_LASTNAME.zip`
-

Exercise 1 (12 points) Ye Olde Item Shoppe — A BST-Powered Shop

Overview: Part A adds BST deletion; Part B adds a range query; Part C builds a small shop simulation on top.

You run a fantasy item shop. The catalog is a `BinarySearchTree<Item>` ordered by price. Customers arrive with a gold budget and a preferred stat; the shop picks the best affordable item, sells it, and drops it from the tree when stock reaches zero.

You first implement two BST methods (Parts A–B); then you use them in the shop (Part C).

Part A — `remove(T value)` on `BinarySearchTree` (4 points)

Add a `remove()` method to your UE7 `BinarySearchTree<T>`. BST deletion has three cases; handle all of them:

1. **No children (leaf):** Replace the node with `null`.
2. **One child:** Replace the node with its only child.
3. **Two children:** Find the *in-order successor* — the leftmost node in the right subtree. Copy its value into the current node, then recursively delete the successor from the right subtree. (The successor has at most one child, so this reduces to Case 1 or 2.)

Part B — `rangeSearch(T min, T max)` on `BinarySearchTree` (3 points)

Add a method that returns all values v in the tree where $\min \leq v \leq \max$ (using `compareTo`):

Part C — Shop Simulation (5 points)

Extend `Item` from UE7 so it can drive shop decisions in the BST. Minimum requirements:

- Keep existing UE7 fields (`name`, `type`, `weight`).
- Add `price` (BST key), `offensive` and `defensive` stats, and `mutable stock`.
- Implement `Comparable<Item>` with deterministic ordering by price (and a stable tie-breaker).
- Provide an operation that reduces stock safely (must not go below zero).

Create `Customer.java` with a `name`, a gold budget (`int`), and a preferred stat. You may choose any representation for the stat.

Create `Shop.java` that wraps a `BinarySearchTree<Item>` and provides at least:

- `restock(Item)` to insert items into the catalog.
- `processSale(Customer)` that:
 - considers only items affordable for the customer's budget (use your BST range query),
 - selects the best affordable item according to the preferred stat,
 - updates stock and removes the item from the BST when stock reaches 0,
 - returns the sold item, or `null` if no sale is possible.

Write a `main()` demo with at least 6 items (varying price / stats / stock) and 5 customers with different budgets and preferences. Print the catalog after each sale.

Exercise 2 (12 points) Dungeon Run Log — Loops vs. Streams

Overview: Part A defines the log and random generator; Part B solves four queries with loops; Part C repeats those four with Streams and adds three more.

Your guild stores dungeon runs: player name, gold earned, rooms visited, items collected, and whether the run was survived.

You will implement the **same queries twice**: first with for-each loops and manual accumulation, then with Streams. The point is to compare style, not to learn seven unrelated APIs.

Part A — DungeonRun and random generator (3 points)

Create `DungeonRun.java` with fields for player, gold earned, rooms visited, items collected, and survived-state. Add two generators:

- one method that creates a single random run from a given name pool and `Random`,
- one method that creates a reproducible log from count and seed.

In your PDF: show the class and print the first 5 runs from `generateLog(20, 42L)`.

Part B — Loop-based queries (4 points, 1 point each)

Constraint (Part B only): use **for-each** loops, **if** statements, and manual accumulation. Do **not** use `stream()`, `Stream`, or `Optional` in these four methods.

1. Filter survived runs.
2. Compute top- n runs by gold (descending, with a consistent tie-break rule).
3. Compute total gold over all runs.
4. Group runs by player.

In your PDF: show all four methods and their output on `generateLog(20, 42L)`.

Part C — Stream rewrites and advanced queries (5 points)

Deliverables

- Rewrite the four Part-B queries with Streams and show that outputs match your loop versions on the same seeded log.
- Add three more stream queries:
 1. average gold per player,
 2. best run per player,
 3. run count per player with alphabetically sorted keys (i.e., map iteration is A–Z).

In your PDF: show all seven stream-side implementations (four rewrites + three new) and their output. For query 7 (sorted counts), add **one sentence** explaining why a `TreeMap` fits better than a `HashMap` for alphabetical order.